

Reto01_Jupyter_Airflow

March 17, 2026

1 Reto 01 - Mini entorno de trabajo con Jupyter Notebook y Apache Airflow

Notebook con los tres ejercicios del reto. He montado el entorno con Docker Compose cogiendo como base la estructura del entorno del profesor (hadoop-lab_v6) y adaptandolo para meter Papermill, la API REST, etc.

El DAG test1 esta sacado del video de Albert Coronado que nos facilito el profesor en clase.

1.1 1. Conexion a la API REST de Airflow

Airflow tiene una API REST en el puerto 8080. Para conectarnos usamos autenticacion basica con usuario `admin` / `admin` que se configura en el docker-compose (en el entrypoint de airflow-init).

```
[1]: import requests
import json
import time

# para quitar el warning de pendulum que sale con airflow 2.8.3
# https://stackoverflow.com/questions/77aborrecido/
↳ suppress-deprecationwarning-pendulum
import warnings
warnings.filterwarnings("ignore", message=".*__version__.*deprecated.*")

# URL del webserver de Airflow (nombre del contenedor en docker-compose)
BASE_URL = "http://reto01-airflow-webserver:8080/api/v1"

# usuario y contraseña del entrypoint del docker-compose
AUTH = ("admin", "admin")

print(f"Conexion configurada a: {BASE_URL}")
print(f"Usuario: admin")
```

Conexion configurada a: `http://reto01-airflow-webserver:8080/api/v1`

Usuario: `admin`

1.2 2. Activar el DAG test1

Antes de lanzarlo hay que activarlo (quitarle el pause). Esto lo hacemos con un PATCH a la API.

```
[2]: # activar el DAG (quitar el pause)
response = requests.patch(
    f"{BASE_URL}/dags/test1",
    json={"is_paused": False},
    auth=AUTH
)

print(f"Estado HTTP: {response.status_code}")
print(f"DAG test1 activado: is_paused = {response.json().get('is_paused')}")
```

Estado HTTP: 200

DAG test1 activado: is_paused = False

1.3 3. Lanzar el DAG test1 con la API REST

Lanzamos el DAG con un POST. En el campo conf le pasamos los parametros que lee la tarea0 del codigo del video de Albert Coronado (el parametro commit).

```
[3]: # lanzar el DAG con commit=000000 (ejecucion normal, sin error)
response = requests.post(
    f"{BASE_URL}/dags/test1/dagRuns",
    json={"conf": {"commit": "000000"}},
    auth=AUTH
)

dag_run = response.json()
print(f"Estado HTTP: {response.status_code}")
print(f"DAG Run ID: {dag_run.get('dag_run_id')}")
print(f"Estado: {dag_run.get('state')}")
print(f"Fecha logica: {dag_run.get('logical_date')}")
```

Estado HTTP: 200

DAG Run ID: manual__2026-03-17T11:48:41.951887+00:00

Estado: queued

Fecha logica: 2026-03-17T11:48:41.951887+00:00

1.4 4. Consultar ejecuciones del DAG

Esperamos un poco a que termine y luego consultamos las ejecuciones con un GET.

```
[4]: # esperamos a que acaben las tareas
print("Esperando 30 segundos...")
time.sleep(30)

# consultar ejecuciones
response = requests.get(
    f"{BASE_URL}/dags/test1/dagRuns",
    auth=AUTH
```

```

)

dag_runs = response.json()
print(f"Total de ejecuciones: {dag_runs.get('total_entries')}")
print("\nUltimas ejecuciones:")
for run in dag_runs.get('dag_runs', [])[-3:]:
    print(f"    - ID: {run['dag_run_id']} | Estado: {run['state']} | Fecha: {run['logical_date']}")

```

Esperando 30 segundos...

Total de ejecuciones: 15

Ultimas ejecuciones:

```

- ID: manual__2026-03-17T11:43:22.821901+00:00 | Estado: success | Fecha: 2026-03-17T11:43:22.821901+00:00
- ID: manual__2026-03-17T11:43:54.255958+00:00 | Estado: failed | Fecha: 2026-03-17T11:43:54.255958+00:00
- ID: manual__2026-03-17T11:48:41.951887+00:00 | Estado: success | Fecha: 2026-03-17T11:48:41.951887+00:00

```

1.5 5. Ver estado detallado de un DAG Run

Miramos el detalle de la ultima ejecucion: estado general y estado de cada tarea.

```

[5]: # cogemos el ultimo dag_run_id
last_run_id = dag_runs['dag_runs'][-1]['dag_run_id']
print(f"Consultando DAG Run: {last_run_id}\n")

# estado general
response = requests.get(
    f"{BASE_URL}/dags/test1/dagRuns/{last_run_id}",
    auth=AUTH
)

run_detail = response.json()
print(f"Estado del DAG Run: {run_detail.get('state')}")
print(f"Inicio: {run_detail.get('start_date')}")
print(f"Fin: {run_detail.get('end_date')}")

# estado de cada tarea
response = requests.get(
    f"{BASE_URL}/dags/test1/dagRuns/{last_run_id}/taskInstances",
    auth=AUTH
)

task_instances = response.json()
print(f"\nTareas ({len(task_instances.get('task_instances', []))}): \n")
for ti in task_instances.get('task_instances', []):

```

```
print(f" - {ti['task_id']}: {ti['state']} (intento {ti.get('try_number', '?')})")
```

Consultando DAG Run: manual__2026-03-17T11:48:41.951887+00:00

Estado del DAG Run: success

Inicio: 2026-03-17T11:48:42.382412+00:00

Fin: 2026-03-17T11:48:45.090153+00:00

Tareas (3):

- tarea0: success (intento 1)
- tarea2: success (intento 1)
- print_date: success (intento 1)

1.6 6. Consultar logs de una tarea

Tambien podemos ver los logs de cada tarea desde la API. Aqui vemos los de `print_date` (el `BashOperator` que imprime la fecha).

```
[6]: # logs de print_date
response = requests.get(
    f"{BASE_URL}/dags/test1/dagRuns/{last_run_id}/taskInstances/print_date/logs/1",
    auth=AUTH,
    headers={"Accept": "text/plain"}
)

print("=== LOGS de print_date ===")
print(response.text[:2000])
```

=== LOGS de print_date ===

69aabdf8dd62

*** Found local files:

*** * /opt/airflow/logs/dag_id=test1/run_id=manual__2026-03-

17T11:48:41.951887+00:00/task_id=print_date/attempt=1.log

[2026-03-17T11:48:44.355+0000] {taskinstance.py:1979} INFO - Dependencies all met for dep_context=non-requeueable deps ti=<TaskInstance: test1.print_date manual__2026-03-17T11:48:41.951887+00:00 [queued]>

[2026-03-17T11:48:44.366+0000] {taskinstance.py:1979} INFO - Dependencies all met for dep_context=requeueable deps ti=<TaskInstance: test1.print_date manual__2026-03-17T11:48:41.951887+00:00 [queued]>

[2026-03-17T11:48:44.367+0000] {taskinstance.py:2193} INFO - Starting attempt 1 of 2

[2026-03-17T11:48:44.384+0000] {taskinstance.py:2217} INFO - Executing

<Task(BashOperator): print_date> on 2026-03-17 11:48:41.951887+00:00

[2026-03-17T11:48:44.391+0000] {standard_task_runner.py:60} INFO - Started process 672 to run task

```
[2026-03-17T11:48:44.397+0000] {standard_task_runner.py:87} INFO - Running:
['***', 'tasks', 'run', 'test1', 'print_date',
'manual__2026-03-17T11:48:41.951887+00:00', '--job-id', '47', '--raw', '--
subdir', 'DAGS_FOLDER/test.py', '--cfg-path', '/tmp/tmpspga81oj']
[2026-03-17T11:48:44.402+0000] {standard_task_runner.py:88} INFO - Job 47:
Subtask print_date
[2026-03-17T11:48:44.486+0000] {task_command.py:423} INFO - Running
<TaskInstance: test1.print_date manual__2026-03-17T11:48:41.951887+00:00
[running]> on host 69aabdf8dd62
[2026-03-17T11:48:44.593+0000] {taskinstance.py:2513} INFO - Exporting env vars:
AIRFLOW_CTX_DAG_EMAIL='test@test.com' AIRFLOW_CTX_DAG_OWNER='***'
AIRFLOW_CTX_DAG_ID='test1' AIRFLOW_CTX_TASK_ID='print_date'
AIRFLOW_CTX_EXECUTION_DATE='2026-03-17T11:48:41.951887+00:00'
AIRFLOW_CTX_TRY_NUMBER='1'
AIRFLOW_CTX_DAG_RUN_ID='manual__2026-03-17T11:48:41.951887+00:00'
[2026-03-17T11:48:44.595+0000] {subprocess.py:63} INFO - Tmp dir root location:
/tmp
[2026-03-17T11:48:44.597+0000] {subprocess.py:75} INFO - Running command:
['/usr/bin/bash', '-c', 'echo "La fecha es $(date)"]
[2026-03-
```

1.7 7. Lanzar el DAG con error controlado

En el código del video, si le pasamos `commit=1` la tarea0 lanza un `AirflowFailException` a propósito. Así se ve como funciona el error controlado y como las tareas que dependen de tarea0 quedan en `upstream_failed`.

```
[7]: # lanzar con commit=1 para que falle a proposito (como en el video)
response = requests.post(
    f"{BASE_URL}/dags/test1/dagRuns",
    json={"conf": {"commit": "1"}},
    auth=AUTH
)

error_run = response.json()
error_run_id = error_run.get('dag_run_id')
print(f"DAG Run ID (error): {error_run_id}")
print(f"Estado inicial: {error_run.get('state')}")

# esperamos
print("\nEsperando 30 segundos...")
time.sleep(30)

# comprobamos que ha fallado
response = requests.get(
    f"{BASE_URL}/dags/test1/dagRuns/{error_run_id}",
    auth=AUTH
)
```

```

error_detail = response.json()
print(f"Estado final del DAG Run: {error_detail.get('state')}")

# vemos el estado de cada tarea
response = requests.get(
    f"{BASE_URL}/dags/test1/dagRuns/{error_run_id}/taskInstances",
    auth=AUTH
)

for ti in response.json().get('task_instances', []):
    print(f" - {ti['task_id']}: {ti['state']}")

```

DAG Run ID (error): manual__2026-03-17T11:49:13.628715+00:00

Estado inicial: queued

Esperando 30 segundos...

Estado final del DAG Run: failed

- tarea0: failed
- print_date: upstream_failed
- tarea2: upstream_failed

1.8 8. CLI de Airflow desde Jupyter

Como en el contenedor de Jupyter tambien esta instalado Airflow (lo puse en el Dockerfile para poder usar la CLI), podemos lanzar comandos directamente con !. Comparte la misma base de datos PostgreSQL que el scheduler asi que ve los mismos DAGs.

```

[8]: # listar DAGs (se ven los mismos que en la web de Airflow)
!airflow dags list 2>/dev/null

```

```

[2026-03-17T11:49:47.920+0000] {plugins.py:37} INFO -
setup plugin alembic.autogenerate.schemas
[2026-03-17T11:49:47.922+0000] {plugins.py:37} INFO -
setup plugin alembic.autogenerate.tables
[2026-03-17T11:49:47.923+0000] {plugins.py:37} INFO -
setup plugin alembic.autogenerate.types
[2026-03-17T11:49:47.923+0000] {plugins.py:37} INFO -
setup plugin alembic.autogenerate.constraints
[2026-03-17T11:49:47.924+0000] {plugins.py:37} INFO -
setup plugin alembic.autogenerate.defaults
[2026-03-17T11:49:47.924+0000] {plugins.py:37} INFO -
setup plugin alembic.autogenerate.comments
dag_id          | filepath          |
owner          | paused
=====+=====+=====+=====
dag_papermill  | dag_papermill.py | airflow | False

```

```
test1          | test.py          | airflow | False
```

```
[9]: # tareas del DAG test1 (sacado del video de Albert Coronado)
!airflow tasks list test1 2>/dev/null
```

```
print_date
tarea0
tarea2
```

```
[10]: # version de airflow (la misma que usa el profesor en su entorno)
!airflow version 2>/dev/null
```

2.8.3

1.9 9. Ventajas de usar Jupyter como cliente de Airflow

- **Pruebas rapidas:** se pueden lanzar DAGs, cambiar parametros y relanzar sin salir del notebook
- **Automatizacion:** con la API REST se puede controlar Airflow por codigo, encadenando lanzamientos y consultas
- **Documentacion:** Jupyter mezcla codigo con texto, asi queda todo documentado paso a paso
- **API vs CLI:** la API funciona por red (entre contenedores), la CLI necesita tener Airflow instalado en la misma maquina. Desde Jupyter se pueden usar las dos

1.10 10. Papermill - ejecutar notebooks con parametros

Papermill permite ejecutar notebooks pasandoles parametros desde codigo. En el `notebook_parametrizable.ipynb` hay una celda con el tag `parameters` que tiene los valores por defecto. Cuando Papermill lo ejecuta, inyecta los valores nuevos encima de esos.

Primero lo probamos en local desde aqui y luego desde un DAG de Airflow.

```
[11]: import papermill as pm

# prueba local: ejecutar el notebook con parametros personalizados
pm.execute_notebook(
    input_path="notebook_parametrizable.ipynb",
    output_path="notebook_output_local.ipynb",
    parameters={
        "nombre": "Alumno",
        "repeticiones": 5
    }
)
print("Notebook ejecutado correctamente. Resultado en notebook_output_local.
↪ipynb")
```

```
Executing: 0%|          | 0/6 [00:00<?, ?cell/s]
```

```
Notebook ejecutado correctamente. Resultado en notebook_output_local.ipynb
```

1.11 11. Ejecutar el notebook desde un DAG de Airflow

El DAG `dag_papermill.py` usa un `PythonOperator` que llama a Papermill para ejecutar el notebook parametrizable. Los parametros se le pasan desde el `conf` del DAG Run, igual que hacíamos con el `test1`.

```
[12]: # activar el DAG de papermill
requests.patch(
    f"{BASE_URL}/dags/dag_papermill",
    json={"is_paused": False},
    auth=AUTH
)

# lanzarlo con parametros
response = requests.post(
    f"{BASE_URL}/dags/dag_papermill/dagRuns",
    json={"conf": {"nombre": "Airflow", "repeticiones": 4}},
    auth=AUTH
)

pm_run = response.json()
print(f"DAG lanzado - Run ID: {pm_run.get('dag_run_id')}")
print(f"Estado: {pm_run.get('state')}")
```

DAG lanzado - Run ID: manual__2026-03-17T11:49:59.096982+00:00
Estado: queued

```
[13]: # comprobar resultado
print("Esperando 30 segundos...")
time.sleep(30)

pm_run_id = pm_run.get('dag_run_id')
response = requests.get(
    f"{BASE_URL}/dags/dag_papermill/dagRuns/{pm_run_id}/taskInstances",
    auth=AUTH
)

for task in response.json().get("task_instances", []):
    print(f"  {task['task_id']}: {task['state']}")

print("\nEl notebook ejecutado se guarda en notebooks/notebook_output.ipynb")
```

Esperando 30 segundos...
ejecutar_notebook: success

El notebook ejecutado se guarda en `notebooks/notebook_output.ipynb`

1.12 12. Conclusiones

- Se ha montado un entorno aislado con Docker siguiendo la estructura del entorno del profesor (hadoop-lab_v6), con Apache Airflow 2.8.3 y Jupyter Notebook
- Desde Jupyter se ha usado Airflow de dos formas: como **cliente** (API REST y CLI) y como **contenido ejecutado** (Papermill)
- El DAG test1 del video de Albert Coronado demuestra PythonOperator, BashOperator, XCom, dependencias entre tareas y error controlado con AirflowFailException
- Con Papermill se pueden meter notebooks como tareas de Airflow sin tener que reescribirlos como scripts